System Calls & System Programs

Topics:
1 System call
2 System Programs



System Calls & System Programs

When a C program wants to read a file from the hard disk, how does it communicate with the Operating System?

What is a System Call?

Definition
A **system call is a programmatic interface through which a user-level program requests a service from the Operating System kernel.**

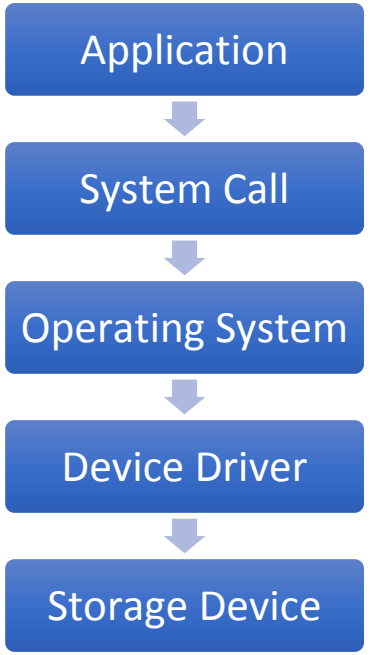
Simple Definition:
System Call = Request made by a program to the Operating System

Examples:
A program may request the OS to:

- Create a process
- Open a file
- Read a file
- Write to a file
- Allocate memory
- Communicate with another process

Why Do We Need System Calls?

Application programs need OS services.
For example, a program wants to: **Read a file**
But the program should not directly control the disk hardware.
Instead:



System calls provide a controlled interface between applications and the OS.

User Mode and Kernel Mode

Modern operating systems generally separate execution into:

User Mode:

Applications execute here.

Examples:

- Chrome
- VS Code
- Media Player

Kernel Mode:

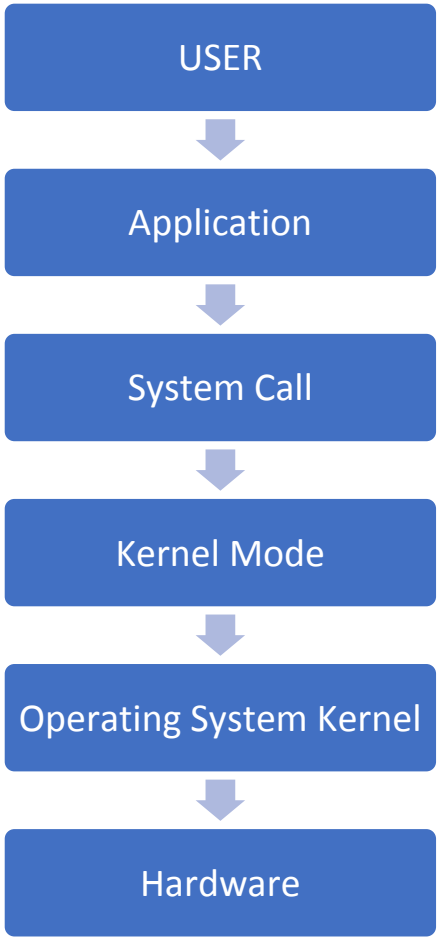
Operating System kernel executes here.

Has privileged access to:

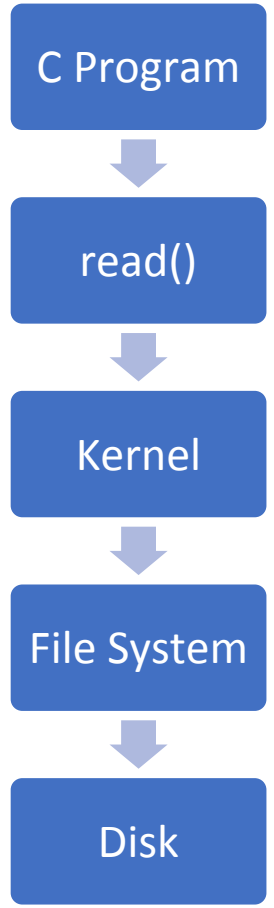
- CPU
- Memory
- Hardware
- I/O devices

System Call Flow

Diagram:



Example:

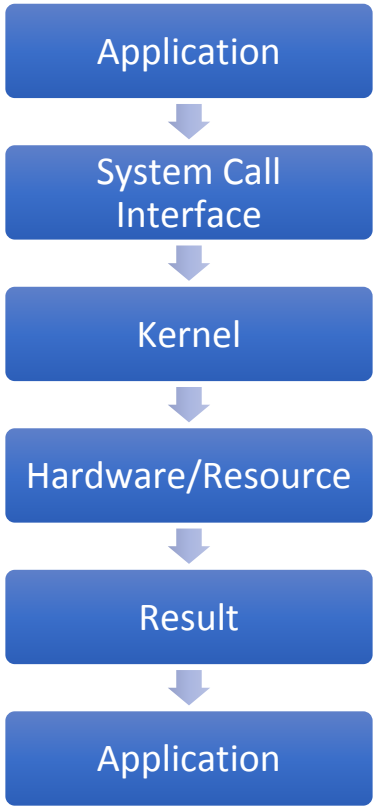


The application doesn’t simply jump into the kernel whenever it wants.
The system call provides a controlled entry point into the operating system.

How Does a System Call Work?

General Flow:

1. Application requests an OS service.
2. System-call interface receives the request.
3. Control transfers to the kernel.
4. Kernel performs the requested operation.
5. Result is returned to the application.



System Call Interface

System Call Interface:
The **System Call Interface (SCI)** provides the mechanism through which applications access OS services.
It:

- Receives system-call requests
- Identifies the requested operation
- Transfers control to the kernel
- Returns the result

Important:
Application → System Call Interface → Kernel

Types of System Calls

According to the standard OS classification, system calls can be grouped into:

1. Process Control
2. File Management
3. Device Management
4. Information Maintenance
5. Communication
6. Protection

Optional modern addition
Some OS designs also expose memory-management operations through system calls.

Process Control System Calls

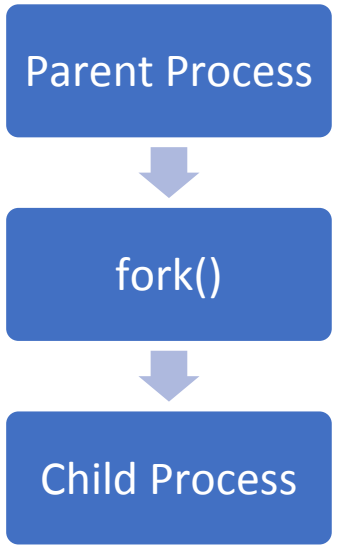
Process Control
Used to manage processes.

Operations:
- Create process
- Terminate process
- Load program
- Execute program
- Wait for process
- Allocate/free memory

Examples in Unix/Linux

```
fork()
exec()
wait()
exit()
```

Example:



File Management System Calls

Used for manipulating files.

Operations:

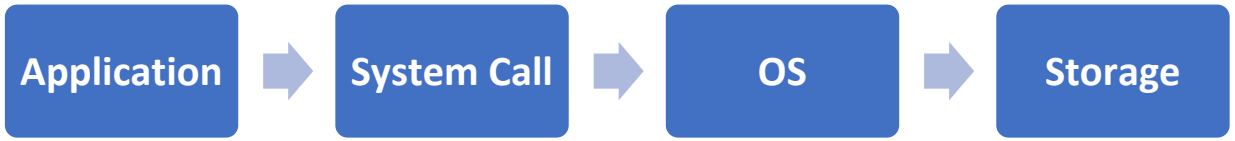
- Create file
- Open file
- Read file
- Write file
- Close file
- Delete file

Common Unix/Linux Examples

```
open()
read()
write()
close()
unlink()
```

Real-Life Example

When a text editor saves a file:



Device Management System Calls

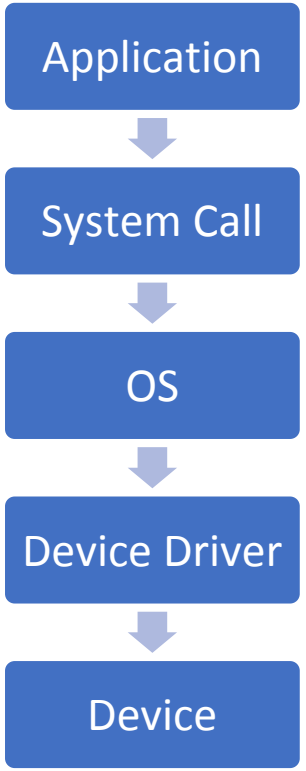
Programs need to communicate with devices.

Operations:

- Request device
- Release device
- Read device
- Write device
- Configure device

Examples:

- Keyboard
- Printer
- Disk
- Network device



Information Maintenance

System calls can be used to obtain or modify system information.

Examples:

- Get system time
- Get process information
- Get system information
- Get/set attributes

Unix/Linux Examples

```
getpid()
time()
uname()
```

Example

A program wants to know:
“What is my process ID?”
It can request the OS for this information.

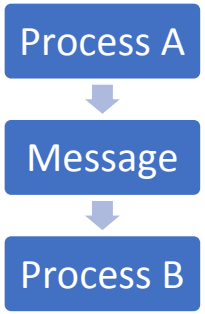
Communication System Calls

Processes often need to communicate.
Two Major Methods:

1 Shared Memory
Processes share a region of memory.

2 Message Passing
Processes exchange messages.

Examples:



Applications

- Web browsers
- Servers
- Databases
- Distributed applications

Protection System Calls

Protection mechanisms control access to resources.

Examples:

- Set permissions
- Get permissions
- Change ownership
- Control access

Example:
A file may have:

Owner → Read + Write
Group → Read
Others → No Access

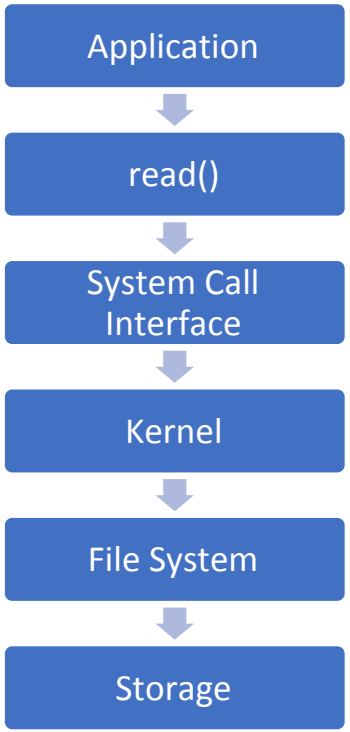
The OS decides **who can access what**.

Summary of System Call Types

TYPE	PURPOSE	EXAMPLES
1 Process Control	Manage processes	fork(), exec(), exit()
2 File Management	Manage files	open(), read(), write()
3 Device Management	Manage devices	Device I/O operations
4 Information Maintenance	System information	getpid(), time()
5 Communication	Process communication	IPC, sockets
6 Protection	Access control	Permission operations

Real Example: read()

Suppose a C program wants to read data from a file.

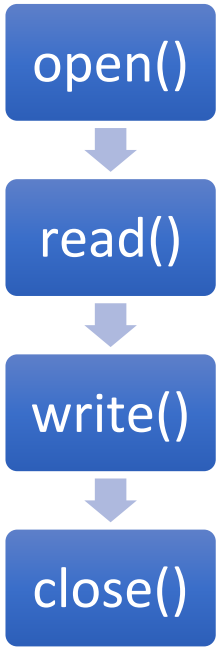


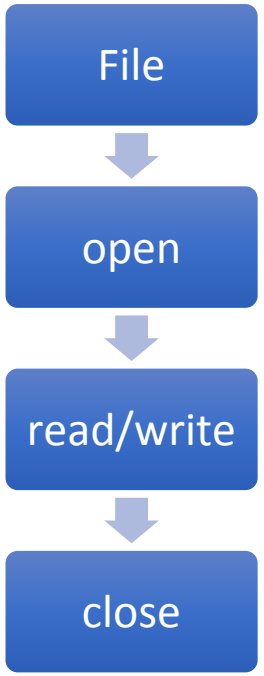
The OS performs the operation and returns the data.

Important:

read() is **not simply a normal user-defined function** when referring to the Unix/Linux system-call interface. It provides access to the kernel's file-reading service.

open(), read(), write(), close()





System Calls vs Normal Functions

SYSTEM CALL	NORMAL FUNCTION
Requests OS service	Performs application/library operation
Enters kernel through system-call mechanism	Usually executes in user space
Controlled by OS	Controlled by program/library
Example: read()	Example: strlen()

Important Clarification:
Every function in a C program is a system call.
False

For example:
printf() is a C library function. It may internally use a system call such as write() to produce output, depending on the implementation.

What are System Programs?

Definition:

- **System programs provide a convenient environment for program development and execution.**
- They are generally utilities that interact with the operating system and help users perform common tasks.

Examples

- File utilities
- Text editors
- Compilers
- Linkers
- Loaders
- Command interpreters
- System utilities

Categories of System Programs

1. **File Management**

- Create files
- Delete files
- Copy files
- Rename files

Examples:

1. cp
2. mv
3. rm

2. Status Information

- Date/time
- System information
- Process information

Examples:

1. date
2. uname
3. ps

System Programs: Communication

Communication Programs:
Allow users/processes to communicate locally or over a network.

Examples:
- Email utilities
- Remote login
- File transfer
- Network utilities

Unix/Linux Examples

ssh
scp
ping

Programming Language Support

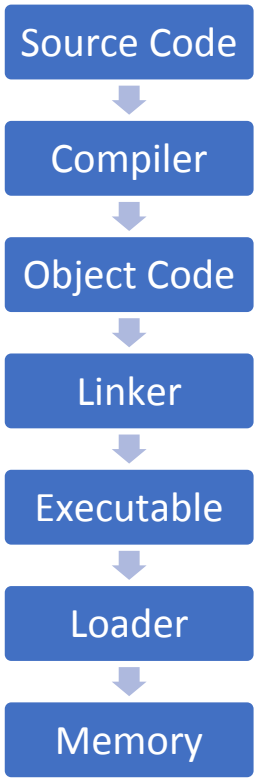
System programs help developers create and execute programs.
Examples:

Compiler
C source → Machine/Executable code

Assembler
Assembly → Machine code

Linker
Combines object files and libraries.

Loader
Loads executable program into memory.



Command Interpreter

Command Interpreter / Shell
A program that accepts commands from the user and executes them.
Examples:
Linux/macOS:

- Bash
- Zsh

Windows:

- Command Prompt
- PowerShell

Example:

```
pwd
ls
cd Documents
```

Important:
Shell is not the kernel.
The shell provides a user interface to interact with the OS.

System Calls vs System Programs

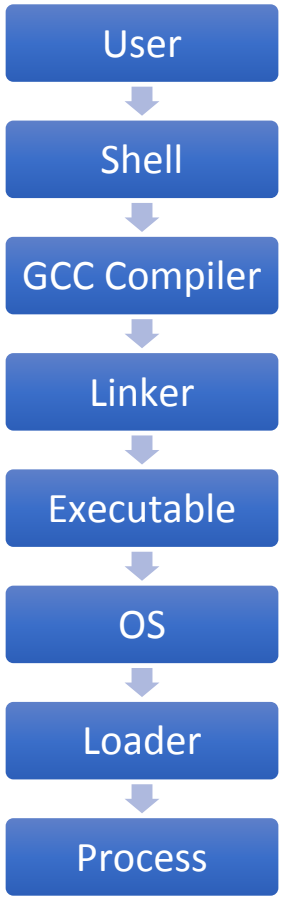
SYSTEM CALLS	SYSTEM PROGRAMS
Interface to OS services	Utilities for users/programmers
Used directly/indirectly by programs	Used to perform common tasks
Lower-level mechanism	Higher-level utilities
read(), write(), fork()	ls, cp, compiler, shell

Easy Way to Remember:
System Call → Request
System Program → Utility

Real-Life Example

Running a C Program
Suppose you execute:

```
gcc program.c -o program
```



This shows how **system programs and OS services work together**.

Quick Quiz

Q1. What is a system call?
A. A hardware device. B. An interface for requesting OS services. C. A programming language. D. A file system

Q2. Which system call is commonly associated with creating a new process in Unix/Linux?
A. open() B. read() C. fork() D. close()

Q3. Which system call is used to read data from a file in Unix/Linux?
A. read() B. fork() C. exec() D. exit()

Q4. Which of the following is a system program?
A. RAM B. CPU C. Shell D. Keyboard

Q5. Which component loads an executable program into memory for execution?
A. Compiler B. Loader C. Text Editor D. Shell

B
C
A
C
B

Summary

System Calls

- Definition
- Need for system calls
- User mode and kernel mode
- System-call interface
- Types of system calls
- Examples

System Programs

- File management utilities
- Status information
- Communication programs
- Compilers
- Linkers
- Loaders
- Shells

Key Concept

Application → System Call → Kernel → Hardware